PHILALINK

Updated Project Specification

A role-based digital healthcare coordination, medication support and clinic management platform

Version 2.0 | October 2026

Prepared by

Syabonga Njabulo Duma

Software Development - Nelson Mandela University

Project type	Full-stack healthcare management platform
Frontend	React + Vite, deployed on Vercel
Backend	ASP.NET Core Web API (.NET 10), deployed on Render
Database	PostgreSQL via Supabase / Npgsql + Entity Framework Core

Portfolio live site: https://philalinkmed.vercel.app/

Project note: PhilaLink is a student-developed health technology platform. It is designed around responsible handling of health-related information but is not a substitute for professional medical care or formal clinical deployment certification.

Document Control

Document	PhilaLink Updated Project Specification
Version	2.0
Last updated	15 October 2026
Scope	Current implemented platform plus clearly identified future roadmap items
Primary audience	Recruiters, lecturers, technical reviewers, project stakeholders and portfolio visitors

Contents

1. Executive Summary
2. Project Origin and Problem Statement
3. Goals, Scope and Success Criteria
4. User Roles and Responsibilities
5. Functional Specification
6. Core Workflows and Business Rules
7. System Architecture and Technology Stack
8. Data Model Overview
9. Security, Privacy and Responsible Health Design
10. Clinic Directory and Facility Data
11. Non-Functional Requirements
12. Testing, CI and Quality Controls
13. Deployment and Operations
14. Current Implementation Status
15. Known Limitations and Future Roadmap
16. Conclusion

Appendix A. Key Backend Modules

1. Executive Summary

PhilaLink is a full-stack healthcare coordination platform that connects patients, caregivers (proxies), nurses, clinic administrators and system administrators through a single role-based application. The platform focuses on practical healthcare workflows such as medication management, repeat medication collection, appointments, patient records, proxy support, clinic discovery, notifications, reporting and administrative oversight.

The current implementation combines a React/Vite frontend with an ASP.NET Core Web API and PostgreSQL database. The system is deployed as a working web application, with the frontend hosted on Vercel and the backend hosted on Render. Database access is handled through Entity Framework Core and Npgsql against PostgreSQL/Supabase.

> **Current product direction**
> PhilaLink has evolved from a patient-focused medication support idea into a broader healthcare coordination platform with five active user roles, multilingual patient experiences, legal acceptance workflows, clinic management, reporting, analytics, audit logging and stronger production-oriented security controls.

2. Project Origin and Problem Statement

2.1 Project origin

The idea for PhilaLink grew from observing how patients in communities can depend on family members, community healthcare workers and clinic staff to keep treatment routines on track. In rural and under-resourced settings, a

missed message, an unclear collection date, transport difficulty or a caregiver not having the right information can disrupt continuity of care.

The project therefore treats healthcare as a connected workflow rather than a single patient screen. A patient may need to know when medication is running out, a proxy may need to help collect medication, a nurse may need to manage collection activity, and clinic or system administrators may need visibility into operational trends. PhilaLink brings those perspectives into one system while keeping access role-specific.

2.2 Problem statement

- Patients may struggle to keep track of medication supply, collection dates, appointments and health-related information across separate channels.
- Caregivers and proxies need a controlled way to support linked patients without receiving unrestricted access to unrelated information.
- Nurses and clinic teams need clearer operational visibility into patients, collections, appointments and medication-related workflows.
- Clinic administrators need staff management, stock visibility, analytics, audit information and exportable reports in one interface.
- System administrators need cross-clinic oversight, clinic management, account administration, reporting and governance controls.
- Language and device barriers can make digital health tools difficult to use if the interface assumes a single language or desktop-only interaction.

3. Goals, Scope and Success Criteria

3.1 Primary goals

- Provide a clear patient experience for medications, appointments, records, clinic discovery and settings.
- Support delegated care through proxy-to-patient relationships with limited, purpose-specific access.
- Provide nurses with practical patient, appointment and collection workflows.
- Provide clinic administrators with clinic-scoped workforce, inventory, reporting and audit capabilities.
- Provide super administrators with system-wide clinic, account, analytics, report and governance tools.
- Apply backend-enforced authorization so the frontend is never treated as the final security boundary.
- Make the patient experience usable in 11 supported written South African languages.
- Keep the application usable across desktop, tablet and mobile screen sizes.

3.2 Scope

The current project scope is a web-based healthcare management and coordination platform. It is not an electronic medical record replacement for a national health system, and it does not claim clinical certification. The implemented platform focuses on patient support, clinic workflows, administrative visibility and secure role-based access.

Out of scope for the current student deployment

Formal integration with national health information exchanges, pharmacy dispensing systems, medical aid claims, laboratory systems, hospital admission systems and regulated clinical decision support would require separate institutional agreements, interoperability work and formal governance review.

4. User Roles and Responsibilities

Role	Primary responsibility
Patient	Manages personal medications, appointments, records, clinic discovery, preferences and patient-facing notifications.
Proxy	Supports one or more linked patients and views permitted patient/collection information

Role	Primary responsibility
	relevant to delegated care.
Nurse	Works with clinic patients, appointments, medication collections and day-to-day care workflows.
Clinic Administrator	Manages clinic-level staff, inventory, reports, analytics and audit information.
Super Administrator	Manages system-wide clinics, clinic administrators, accounts, analytics, reports and governance.

5. Functional Specification

5.1 Patient / citizen capabilities

- Account registration with guided patient onboarding and cellphone verification.
- Secure login, password recovery, Google sign-in support and forced password-change flows where required.
- Legal acceptance workflow for Terms of Use and Privacy Policy before protected application access.
- Dashboard with patient-focused health and medication information.
- Medication schedules, medication logs, supply tracking and collection-related information.
- Appointment booking and rescheduling within the configured booking policy.
- Personal health records, allergies, medical conditions and supported health metrics.
- Nearest-clinic / clinic finder experience using facility location data.
- Notifications and reminders for supported healthcare events.
- Theme and language preferences.
- Philani healthcare assistant for supported informational interactions, with the assistant positioned as supplementary rather than a replacement for professional care.

5.2 Proxy capabilities

- Role-specific dashboard for delegated care responsibilities.
- View patients linked to the proxy relationship.
- View relevant patient and medication-collection information permitted by the system.
- Track collection-related activity for linked patients.
- Maintain proxy account settings without exposing unrelated patient data.

5.3 Nurse capabilities

- Clinic workload dashboard and patient search/filtering.
- Patient list and patient-care views.
- Appointment management views for clinic workflow.
- Medication collection management and collection processing.
- Access to relevant clinic and patient operational information according to nurse authorization.
- Role-specific settings and session controls.

5.4 Clinic Administrator capabilities

- Clinic analytics dashboard with operational summaries.
- Clinic staff management and staff registration workflows.
- Clinic inventory / medication stock management.
- Report builder with preview, filters and export support.
- Clinic-scoped audit log access.
- Clinic-level operational visibility without super-admin-only system management privileges.

5.5 Super Administrator capabilities

- System overview across participating facilities.
- System analytics and system-level reporting.
- Account management for supported roles.
- Clinic directory management and clinic activation/deactivation workflows.
- Clinic administrator registration and management.
- System audit visibility and governance-oriented oversight.

6. Core Workflows and Business Rules

6.1 Patient onboarding

1. A user begins patient registration and provides the required identity/contact information.
2. The platform validates the submitted information and completes cellphone verification using an OTP workflow.
3. The patient selects or associates with a clinic during registration where required by the flow.
4. After authentication, protected access is gated by legal-document acceptance when a current Terms of Use or Privacy Policy requires acceptance.
5. The authenticated patient is routed to the patient dashboard and role-specific application layout.

6.2 Appointment booking window

PhilaLink applies a rolling 14-day appointment booking and rescheduling window using South African local time as the authoritative calendar context. The same concept is reflected in the frontend user experience and backend validation.

Rule	Expected behaviour
Past calendar date	Rejected.
Today	Allowed only when the selected time is still in the future.
Tomorrow through +14 days	Allowed.
Beyond +14 calendar days	Rejected.
Timezone	Africa/Johannesburg / South Africa Standard Time is used for booking-window interpretation.

Why the backend rule matters

The frontend prevents invalid selections for usability, but the API also validates the appointment window so a user cannot rely on bypassing the browser interface to submit an unrestricted booking.

6.3 Medication collection workflow

1. Medication and supply information is associated with the patient.
2. The system tracks medication schedules, logs and collection information.
3. The patient and permitted proxy can view upcoming collection-related information.
4. Nurses manage collection activity through clinic workflow screens.
5. Administrators can analyse and report on collection-related activity at the appropriate scope.

6.4 Language preference workflow

The current patient experience supports 11 written languages: English, isiZulu, isiXhosa, Afrikaans, Sepedi, Setswana, Sesotho, Xitsonga, siSwati, Tshivenda and isiNdebele. Language selection is normalised to supported language codes and is retained as a user preference. The translation architecture merges shared resources with patient-page and feature-specific translation resources.

Accessibility note

South African Sign Language is not part of the current text-translation implementation. Future accessibility work can extend beyond text translation into sign-language, screen-reader and simplified-content support.

7. System Architecture and Technology Stack

7.1 High-level architecture

CLIENT
React + Vite + React Router + Tailwind utilities + i18next
V
API
ASP.NET Core Web API (.NET 10) with role-based controllers and services
V
DATA ACCESS
Entity Framework Core + Npgsql
V
DATABASE
PostgreSQL / Supabase
V
EXTERNAL SERVICES
Google authentication flow, Gemini chatbot provider, OpenWeather API where used

7.2 Frontend technology

- React 18 with component-based, role-specific pages and layouts.
- Vite build tooling and production bundling.
- React Router for public routes, authenticated routes, role routes and legal/password gates.
- i18next/react-i18next for language resources and patient localization.
- Leaflet/react-leaflet for map/location experiences.
- Recharts for analytics visualisation.
- Vitest and Testing Library for frontend testing.

7.3 Backend technology

- .NET 10 ASP.NET Core Web API.
- Entity Framework Core 10 with Npgsql PostgreSQL provider.
- JWT Bearer authentication with strict lifetime validation and zero clock skew.
- BCrypt password hashing.
- Swagger/OpenAPI for API inspection and development.
- Dependency-injected services for authentication, patients, nurses, clinics, appointments, medication, collections, notifications, audit, chatbot and legal documents.

8. Data Model Overview

The database is organised around identity/profile records, clinic operations, patient health information, medication/collection workflows, proxy relationships, communication/security and legal acceptance records.

Domain	Key entities
Identity / profiles	User, Patient, Nurse, Proxy, Admin
Clinics	Clinic, ClinicStock
Patient health	Allergy, MedicalCondition, SymptomAssessment, HealthMetric, HealthRecord, PatientPreference

Domain	Key entities
Appointments	Appointment
Medication	Medication, MedicationSchedule, MedicationLog
Collections	MedicationCollection, MedicationCollectionItem
Delegated care	ProxyLink
Communication / security	Notification, AuditLog, OtpVerification, ChatConversation, ChatMessage
Legal / consent	LegalDocument, UserLegalAcceptance

8.1 Relationship principles

- A single User record provides identity and role information while role-specific profile entities store additional domain data.
- Clinic associations scope nurse, clinic-admin and patient-related operational views where applicable.
- ProxyLink records represent delegated-care relationships instead of granting a proxy unrestricted access to the entire patient population.
- Medication schedules/logs and collection entities are separated so daily adherence and collection workflow can evolve independently.
- Audit logs and legal acceptances are stored as independent records to support accountability and traceability.

9. Security, Privacy and Responsible Health Design

Because PhilaLink handles health-related and identity information, security controls are treated as backend responsibilities rather than purely visual restrictions. The implementation includes several production-oriented controls, while formal clinical or regulatory certification remains outside the scope of the student deployment.

Control area	Current approach
Authentication	JWT Bearer tokens; patient OTP verification; password recovery; Google authentication support.
Authorization	Role policies for Patient, Proxy, Nurse/clinic staff, ClinicAdmin and SuperAdmin workflows.
Session revocation	TokenVersion checks allow security-sensitive account changes to invalidate older JWT sessions.
Password storage	BCrypt hashing; raw passwords are not stored.
Transport / hosting controls	HTTPS-oriented deployment, HSTS configuration and forwarded-header handling for hosted reverse proxies.
Request hardening	Configured request body limits, rate limiting, restricted CORS origins and removed Kestrel server header.
Auditability	AuditLog records and admin-facing audit workflows.
Legal acceptance	Versioned legal documents and explicit user acceptance records.
Frontend protection	Protected routes, role routes, legal acceptance gates and forced password-change routes.

9.1 POPIA-aligned design considerations

- Purpose-specific access: each role receives only the functions relevant to its responsibilities.
- Accountability: audit information and acceptance records create a traceable operational history.
- Security safeguards: backend authorization, password hashing, session revocation and rate limiting reduce avoidable exposure.
- Transparency: Terms of Use and Privacy Policy are presented through an explicit legal acceptance flow.
- Data quality: clinic/facility information can be reviewed and maintained rather than being permanently hard-coded into the interface.

Important wording

The project is designed with POPIA principles in mind, but this specification does not claim that PhilaLink has undergone a formal legal compliance audit or clinical security certification.

10. Clinic Directory and Facility Data

PhilaLink includes a clinic/facility directory that supports patient clinic discovery and administrator management. Facility records can contain name, type, address, contact number, coordinates, services, operating hours and active status. The administrator interface supports search, filtering, editing and activation/deactivation workflows.

- Facility coordinates support proximity-oriented clinic discovery.
- Address, contact and operating-hours fields are maintained separately so incomplete public-source data does not require invented information.
- The current database has been expanded beyond placeholder facilities using real South African facility data and administrative cleanup workflows.
- Clinic and hospital hours can be displayed in the admin directory and patient-facing discovery experiences where populated.
- Facility data should be treated as maintainable reference data because public healthcare directories, telephone numbers and hours can change over time.

Data-quality rule

Where a facility detail cannot be confidently verified, the preferred approach is to keep the value empty or use a clearly identified locality fallback rather than fabricate a street address or contact number.

11. Non-Functional Requirements

Quality attribute	Requirement
Security	Sensitive actions must be validated and authorized on the backend; secrets must not be committed to frontend source.
Usability	Role-specific navigation, clear status labels, loading states, forms and mobile-friendly layouts.
Responsiveness	Core features should remain usable on desktop, tablet and mobile.
Performance	API calls should avoid unnecessary payloads; database queries should use indexes, projection and pagination where appropriate.
Reliability	Database connections use retry support; role/business rules are validated server-side.
Maintainability	Separate frontend pages/components/services and backend controllers/services/data models.
Auditability	Important account and operational activity should be traceable through audit workflows.
Localization	Patient-facing text should use shared translation resources rather than hard-coded language-specific copies.

12. Testing, CI and Quality Controls

Both repositories include automated quality tooling. The frontend exposes build, lint and Vitest commands, while the backend contains a separate xUnit test project. GitHub Actions workflows are used to validate changes before merge.

- Frontend: ESLint, Vite production build, Vitest and Testing Library.
- Backend: dotnet restore/build/test with a dedicated xUnit test project.
- Pull-request workflow: feature branches are preferred so CI can run before changes reach the default branch.
- Dependency security: NuGet restore is configured to surface vulnerable dependencies and the project can update affected packages before merge.

- Responsive QA: role dashboards and navigation are tested across desktop/tablet/mobile layouts, including mobile sidebar behaviour.

13. Deployment and Operations

Area	Current deployment approach
Frontend hosting	Vercel - React/Vite production deployment
Backend hosting	Render - ASP.NET Core API / container-oriented hosting
Database	Supabase-hosted PostgreSQL
API documentation	Swagger/OpenAPI in supported environments
Source control	GitHub repositories with pull-request and CI workflows
Configuration	Environment-specific secrets and connection strings are supplied outside committed frontend source.

Operational caution

The production student deployment is suitable for demonstrations and portfolio review. A real clinical rollout would require stronger operational processes such as formal incident response, backup/recovery verification, data retention rules, penetration testing, privacy impact assessment and institutional ownership.

14. Current Implementation Status

Capability	Status
Patient registration + OTP verification	Implemented
Secure login + password reset	Implemented
Google authentication flow	Implemented
Legal Terms/Privacy acceptance gate	Implemented
Patient dashboard	Implemented
Medication tracking and schedules	Implemented
Medication collection workflow	Implemented
Patient appointments	Implemented
14-day appointment booking/rescheduling window	Implemented
Patient records / health information	Implemented
Clinic finder / facility directory	Implemented
11 written-language patient interface	Implemented
Philani healthcare assistant	Implemented
Proxy dashboard and linked-patient views	Implemented
Nurse dashboard / patient / appointment / collection flows	Implemented
Clinic Admin analytics, staff, inventory, reports, audit	Implemented
Super Admin analytics, clinics, clinic admins, reports, accounts, audit	Implemented
PDF / Excel administrative report export	Implemented
Responsive role-based navigation	Implemented
Automated CI workflows	Implemented

15. Known Limitations and Future Roadmap

15.1 Current limitations

- The application is a student-built health technology platform and has not completed formal clinical certification or external penetration testing.
- Public facility data can become outdated; contacts, hours and addresses need ongoing verification and administrative maintenance.
- The current localization implementation covers 11 written languages but does not yet provide South African Sign Language support.
- The system is web-based and depends on network access; offline-first workflows are not yet a core feature.
- The appointment model currently focuses on a rolling booking window rather than advanced capacity/slot optimisation across providers and rooms.

15.2 Future roadmap

Roadmap area	Planned direction
Offline / low-data mode	Improve use in areas with intermittent connectivity through cached essential data and queued actions.
Advanced appointment scheduling	Introduce slot capacity, provider availability, conflict detection and configurable clinic calendars.
Interoperability	Explore standards-based integration such as FHIR where institutional access and governance permit.
Notifications	Extend beyond in-app notifications to controlled SMS/email/push workflows with consent and delivery tracking.
Accessibility	Expand screen-reader testing, simplified-content modes and investigate SASL-support approaches.
Security assurance	Formal threat modelling, penetration testing, privacy impact assessment and stronger operational monitoring.
Facility data operations	Scheduled verification of facility contact details, hours and closures from trusted health department sources.
Clinical governance	Introduce clinician-reviewed content governance if the healthcare assistant or symptom features are expanded.

16. Conclusion

PhilaLink demonstrates a complete software-development lifecycle across product discovery, data modelling, API development, frontend engineering, deployment, security hardening, responsive design, multilingual support and ongoing maintenance. The project is strongest when viewed not as a single dashboard, but as a connected role-based healthcare workflow that considers patients, caregivers, clinic staff and administrators together.

The current implementation is substantial enough to operate as a live portfolio system while still leaving clear room for future clinical governance, interoperability, accessibility and operational maturity. That balance makes PhilaLink both a practical software project and a platform that can continue evolving beyond its original medication-support concept.

Appendix A. Key Backend Modules

Area	Representative backend modules
Authentication / identity	AuthController, AccountIdentityController, PasswordResetController
Patient	PatientController
Appointments	AppointmentsController
Medication	MedicationController, MedicationSupplyController

Area	Representative backend modules
Collections	CollectionsController
Proxy	ProxyController, ProxyCareController, ProxyProfileController
Nurse	NursesController
Clinics / stock	ClinicController, ClinicStockController, RegistrationClinicController
Admin	AdminController, ClinicAdminController, SuperAdminController
Reports	ClinicAdminReportsController, ClinicAdminSecureReportsController, SuperAdminReportsController
Audit / notifications	AuditController, NotificationController, MyNotificationsController
Preferences / legal	LanguagePreferenceController, ThemePreferenceController, LegalDocumentsController
Health support	SymptomAssessmentController, ChatbotController, WeatherController

Specification basis

This version reflects the current PhilaLink frontend/backend repository structure and the implemented project direction as of October 2026. Future changes to routes, database entities, facility data, external integrations or security controls should trigger a corresponding specification update.